

Dimensionality Reduction

Let our data be a matrix X of size $n \times d$, where n is the number of samples and d is the number of features. This is a common and flexible way to represent data for a machine learning problem. With this structure, every row is a sample and every column is a feature.

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}$$

The first index is the data point, the second index is the feature. Dimensionality reduction aims to reduce the number of features d to a smaller number k . The main reason for doing this is that fewer features are easier to reason about, especially for humans but also for machines.

Let's say we pick a completely random subset of the features. – Illustrate with MNIST example. Show half-corrupted images...

Let's say we look at the variance within each feature. – Illustrate with MNIST example. Show variance of each pixel and what it looks like to remove the lowest variance pixels.

Principal Component Analysis

Principal Component Analysis (PCA) is a fancy sounding name. However, the main idea is simple; before removing features, we rotate the feature space. There are many other interpretations, but let's reason from this perspective for now. Our new rotated "features" will be linear combinations of the original features. We want the rotated features to capture more of the variance using fewer features. Otherwise there is no point in rotating the feature space. How do we find a rotation that gets the most variance in the fewest features?

Any rotation can be described by a matrix. However, not all matrices describe rotations. What properties must a matrix have to describe a rotation? Intuitively, if we take an orthonormal basis and map it to another orthonormal basis, we have performed a rotation (and perhaps a reflection too, but ignore that for now). What should a matrix look like to map one orthonormal basis to another when expressed in the original basis? Every basis vector e_i is mapped to a new basis vector e'_i through the rotation matrix R : $e'_i = Re_i$. The new basis vectors are the columns of R . Since the new basis must be orthonormal, the columns of R must be orthonormal. This means that $R^T R = I$.

To exclude reflections, we also require $\det(R) = 1$. If a reflection is involved, such as with $R = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$, then $\det(R) = -1$.

Okay, we now know what a rotation looks like and how to apply it. How do we find the right one? One way is to optimize the rotation matrix greedily, one column at a time. Think about what it would look like to apply the rotation to our data matrix:

$$X' = XR$$

The rows (data points) of X are projected onto the column space of R . See my post about quickly interpreting matrix multiplication for more details on this. Let's maximize the variance for the projections onto the first column e'_1 . The product Xe'_1 is a column-vector of length n where each element i is the projection of the i th data point's features onto the line spanned by e'_1 . The variance of Xe'_1 is given by the standard formula for variance:

$$\frac{1}{n}(Xe'_1 - \overline{Xe'_1})^T(Xe'_1 - \overline{Xe'_1})$$

where $\overline{Xe'_1}$ is the mean of the vector Xe'_1 . Before performing PCA, one should normalize the data. Normalize means to subtract the mean and divide by the standard deviation for each feature. After normalization, each feature has mean 0 and standard deviation 1. Setting the standard deviation to 1 is not necessary for the math to work out, but it's a good idea in practice. The reason is that features with larger values will dominate the objective function.

With 0 mean, we can drop the $\overline{Xe'_1}$ term:

$$\max_{e'_1} \left[\frac{1}{n}(Xe'_1)^T(Xe'_1) \right]$$

We can solve this optimization problem analytically! Ignore the $\frac{1}{n}$ term since it doesn't affect the solution and use $(Xe'_1)^T = e'^T_1 X^T$:

$$\max_{e'_1} [e'^T_1 X^T X e'_1]$$

A matrix $X^T X$ is symmetric. This is because $X^T X = (X^T X)^T = X^T X$. Symmetric matrices can be diagonalized in an orthonormal basis. I won't explain why this is true here, but lookup the spectral theorem for details. Let v_i be the projection of e_1 onto the i th eigenvector of $X^T X$. We can then express e'_1 as a sum of these projections:

$$e'_1 = \sum_{i=1}^d v_i$$

plugging this in, the objective simplifies to:

$$\max_{e'_1} \left[\left(\sum_{i=1}^d v_i \right) \lambda_i \left(\sum_{i=1}^d v_i \right) \right]$$

Since the eigenvectors are orthonormal, we have $v_i^T v_j = \delta_{ij}$. Therefore, the objective simplifies to:

$$\max_{e'_1} \left[\sum_{i=1}^d \lambda_i v_i^T v_i \right]$$

The maximum is achieved when $v_i^T v_i = 1$ for the largest eigenvalue and $v_i^T v_i = 0$ for all other eigenvalues. We can conclude that e'_1 is simply the eigenvector corresponding to the largest eigenvalue of $X^T X$.

It's natural at this point to guess that the next column e'_2 should be the eigenvector corresponding to the second largest eigenvalue of $X^T X$. This is correct! I won't dive into the details here, but one can subtract the component of e'_1 from each data point and apply the same variance maximization with the remaining values. We can repeat this process for all k columns. The resulting rotation matrix R will have columns that are the top k eigenvectors of $X^T X$.

After performing the rotation, we can drop the features with the lowest variance until we reach the desired dimensionality k . This is equivalent to keeping the top k eigenvectors of $X^T X$.

0.1 Autoencoders

When thinking about PCA as a projection, we can ask ourselves "Why require the projection to be linear? Can't we find a non-linear projection that captures more variance?" This is the motivation behind autoencoders.